

Evaluating Context Access Strategies for Repository-Level LLM Agents in Software Engineering Tasks

Durga Susmitha Chakka

Graduate Seminar, Department of Computer Science
Texas A&M University
San Antonio, TX, USA
dchak01@jaguar.tamu.edu

Shivani Sutrave

Graduate Seminar, Department of Computer Science
Texas A&M University
San Antonio, TX, USA
ssutr01@jaguar.tamu.edu

Abstract—Large language models (LLMs) have shown strong potential in software engineering tasks such as bug repair, test-driven correction, and code generation. However, when these tasks are performed at the repository level, model performance does not depend only on the capability of the LLM itself, but also on how repository context is provided during reasoning and code modification. Static prompt injection, retrieval-augmented generation (RAG), and structured tool-based interaction represent three common approaches to this problem, but are rarely compared under the same experimental conditions.

This paper presents a controlled evaluation framework to compare these three context-access strategies in repository-level software engineering tasks. The study is conducted on a Python service repository with deterministic test-based validation and covers three task categories: controlled fault-injection repair, failure test correction, and feature-driven implementation. Across all conditions, the same underlying model configuration, execution environment, and stopping criteria are maintained in order to isolate context-access strategy as the primary experimental variable.

The evaluation focuses on effectiveness, efficiency, and reliability. In addition to task success rate, the framework measures iteration count, runtime, token usage, and behavioral indicators such as invalid file references, failed patch applications, and unnecessary edits. Rather than assuming that one approach is superior, the goal of this work is to characterize the tradeoffs between static prompting, retrieval-based context access, and structured tool mediation for repository-level LLM agents.

Index Terms—large language models, software engineering agents, retrieval-augmented generation, Model Context Protocol, repository-level code repair, tool-augmented LLMs, empirical evaluation

I. INTRODUCTION

Over the past few years, large language models (LLMs) have shown significant progress in software engineering tasks such as code generation, debugging, and automated program repair. Systems based on models like Codex [1] demonstrated that LLMs can produce functional code solutions when the required context is clearly provided. However, most early evaluations were performed on isolated coding problems where the entire task context is available in a single prompt.

Real-world software development environments are considerably more complex. Modern codebases typically span multiple

modules and contain hundreds or thousands of interconnected functions. When a bug occurs, developers must first identify which files are relevant before they can diagnose and repair the issue. This process of navigating repository structure and locating relevant context is an important part of software maintenance.

Several approaches have emerged for providing repository context to LLM-based coding agents. One common approach is prompt-based context injection, where selected source files are directly placed into the model prompt. Another approach is retrieval-augmented generation (RAG) [4], which retrieves relevant code fragments from a repository index before each generation step. More recently, tool-based interaction mechanisms have been proposed, where an agent can dynamically explore the repository using tools such as file readers, search utilities, or test execution commands. In this study, the tool-based condition is implemented using a Model Context Protocol (MCP) server that provides structured access to repository operations.

These approaches represent different ways of exposing repository context to language models. Each strategy provides distinct advantages and limitations in terms of context availability, efficiency, and system complexity. Understanding how these mechanisms influence agent behavior is therefore important for designing reliable LLM-based software engineering systems.

This paper presents an experimental framework for studying repository-level LLM agents under controlled conditions. Using a Python service repository with test-driven validation, we evaluate three context-access strategies across multiple software engineering tasks, including bug repair, failing-test correction, and feature implementation. The objective of the study is to analyze how different context-access mechanisms influence agent behavior when operating on real code repositories.

Contributions:

- 1) A reproducible evaluation harness where every variable except context-access strategy is fixed across runs.

- 2) A controlled repository-level benchmark covering fault-injection repair tasks, failing-test correction scenarios, and feature-driven implementation tasks across the same experimental framework.
- 3) A metric set that goes beyond pass rate to capture iteration cost, token usage, and behavioral failure modes.
- 4) Four hypotheses derived from previous studies help structure the experimental analysis.

II. PROBLEM STATEMENT

Despite rapid advances in LLM-based coding assistants, there is still limited controlled research on how different context-access mechanisms affect agent performance in real software repositories. Most existing work evaluates the model itself rather than the method used to provide repository context.

As a result, it remains unclear whether performance differences in repository-level software engineering tasks are caused primarily by the underlying language model, by the context-access strategy, or by the interaction between the two. This gap is especially important in multi-file repositories, where relevant information may be distributed across several modules and where incomplete or poorly structured context can lead to inefficient reasoning, invalid edits, or failed repairs.

This study addresses that problem by evaluating three context-access strategies: prompt-only context injection, retrieval-augmented generation, and structured tool-mediated interaction, under the same model configuration and controlled execution environment.

III. RESEARCH QUESTIONS

We are not trying to prove that any strategy is better. The question that actually interests us is more conditional: does the choice of context-access mechanism matter at all, and if so, for which kinds of tasks? With that in mind, we organized the study around four questions.

RQ1 - Success Rate. How do prompt-only, retrieval-augmented generation (RAG), and tool-mediated interaction (MCP) compare in terms of test-pass success rate across controlled fault-injection tasks?

RQ2 - Efficiency. How do the three strategies differ in the cost of reaching a successful solution, measured through iteration count, runtime per task, and total token usage?

RQ3 - Reliability and Hallucination. Do structured tools (MCP) reduce unreliable agent behaviors such as invalid file references, failed patch applications, or edits to unrelated modules?

RQ4 - Task Sensitivity. Do different context-access strategies perform differently depending on task characteristics such as fault locality, cross-module dependencies, or feature implementation complexity?

IV. RELATED WORK

A. LLMs for Code Generation and Bug Repair

The Codex paper [1] is where most discussions of LLM code evaluation start, and for good reason, it introduced

pass@k, which remains the standard way to measure whether a generated fix actually works. But the tasks in that evaluation were single functions with all the needed context already provided. Nothing in the benchmark requires the model to figure out which part of a larger system is relevant. Our work uses the same pass@k metric but applies it to a setting where context navigation is the central challenge.

Tufano et al. [2] took a different angle, mining bug-fixing commits from open-source projects and training neural translation models to reproduce the fix. Their main contribution for our purposes is methodological: they showed that real test suites make reliable evaluation and the diffs are a natural output format for repair systems. We rely on both of these design choices in our setup.

Jiang et al. [3] introduced the idea of iterative self-repair, where the model sees the test output after each attempt and tries again. Fix rates went up, but something interesting also showed up in the failure cases: agents that did not have the right context kept making the same wrong edits in circles. That observation is part of why we track iteration count and repeated ineffective actions as first-class metrics.

B. Retrieval-Augmented Generation for Code

Gao et al. [4] provide a broad survey of retrieval-augmented generation for large language models and summarize the general retrieve-then-generate pattern used to improve grounding and reduce irrelevant context. That pattern maps naturally onto the code repair problem, where failing test output can be used to retrieve repository context before generation.

Zhang et al. [5] extend this idea to repository-level code tasks through iterative retrieval and generation. Their findings are directly relevant to our setting: retrieval quality depends strongly on chunking decisions, ranking quality, and whether the retrieved context actually covers the fault location. We therefore use a deterministic baseline for RAG so that the same query returns the same ranked chunks across runs. This keeps the comparison interpretable and reduces experimental variance from the retrieval layer itself.

C. Tool-Augmented LLM Agents

Toolformer [6] demonstrated that language models can learn to call external APIs during generation, which helps reduce factual errors. In that work, API calls are inserted directly within the generation process. Our approach differs in that, tool interaction is handled through a separate MCP server that validates and executes tool requests. Despite this difference in implementation, the core motivation is similar: grounding model outputs through interaction with external tools can reduce the hallucinations that often make LLM-generated code unreliable.

ReAct [8] is probably the closest architectural ancestor of our MCP agent loop. The idea of alternating between a reasoning step and a tool call, then feeding the tool’s output back into the next reasoning step, is essentially how Strategy C in our setup works. ReAct was evaluated on question-answering tasks, but the loop structure transfers well to bug

repair, where each `read-file` or `run-tests` call gives the agent new information to reason about.

Gorilla [7] is relevant for a specific reason. Patil et al. showed that when tool schemas are well-specified, models make substantially fewer malformed API calls. That finding is part of why we built the MCP server where every tool call gets checked before execution, which means invalid calls show up in the logs as a concrete failure signal rather than silently producing garbage.

WebGPT [9] is less directly related but methodologically important. Nakano et al. argued that for tool-using agents, tracking the sequence of actions matters as much as the final answer. An agent that gets to the right answer through five unnecessary browser requests is not the same as one that does it in two. We apply the same logic here: it is not enough to know whether the bug got fixed; we want to know how many tool calls it took and whether any of them were wasteful.

D. Evaluation Benchmarks

SWE-bench [10] is the benchmark that our work is most directly in conversation with. It asks models to resolve actual GitHub issues, evaluated against the repository’s own test suite, the same test-driven setup we use. The difference is that SWE-bench uses real, heterogeneous repositories, which makes it hard to attribute performance differences to any single factor. We trade ecological validity for control: a synthetic repo with known faults lets us say something definitive about whether context-access strategy matters.

AgentBench [11] evaluated tool-using agents across a variety of tasks and emphasized that step count and error behavior are as informative as success rate. That shaped how we designed our metric collection. Knowing an agent succeeded but needed fifteen iterations and hallucinated three file paths along the way is useful information that a binary pass/fail score would hide.

To the best of our knowledge, previous work has not provided a controlled comparison in which prompt injection, RAG, and tool-mediated access are evaluated on the same repository-level tasks under a shared model and execution setting. That is the specific gap this study targets.

V. EXPERIMENTAL DESIGN

A. Target Repository

We needed a codebase that was realistic enough to pose genuine context-navigation challenges but controlled enough that we could inject known faults and measure outcomes cleanly. A Python service application with approximately 7,000 lines of code fit those requirements. The repository has separate modules for API routing, authentication, business logic, database access, security, and configuration, a structure common in real backend services and one where a bug in one layer can only be diagnosed by reading another.

All `pytest` tests pass on the clean codebase, and the repository is Git-initialized. The Git setup matters specifically for Strategy C: the MCP server’s `apply_patch` tool calls

`git apply` internally, and `git_diff` gives the agent a view of what it has changed so far.

B. Task Construction

The experimental design covers three task categories. The first category consists of controlled fault-injection tasks, where logical bugs are introduced via patch files applied by `scripts/apply_task.py`. These faults vary in type such as off-by-one errors, incorrect conditional logic, missing validation, and wrong imports. Some are contained within a single function, while others require reading across module boundaries.

The second category consists of failing-test correction scenarios. In these tasks, additional tests are introduced to expose missing edge-case handling, validation failures, or behavioral inconsistencies. The agent must identify the underlying cause of the failing test and modify the repository so that the entire suite passes.

The third category consists of feature-driven implementation tasks. Here, new functionality is specified through added tests, and the agent must implement the required behavior so that those tests pass without breaking existing functionality.

The repository is restored to its clean state before each run using `scripts/reset_repo.py`, ensuring that every strategy is evaluated from the same baseline state.

C. Context-Access Strategies

Strategy A (Prompt-Only). We provide the model with a static snapshot of repository context selected to be relevant to the task, together with the task description and test output. The model sees that context directly in the prompt and has no way to request additional repository information during a run. This serves as the baseline condition.

Strategy B (RAG). At the start of each run we build an index over all repository source files. Before each generation step, the failing test output is used as a query and the top- K chunks are retrieved and injected into the prompt. The model never reads a full file directly, it only sees what retrieval surfaces.

Strategy C (MCP). In this strategy, the agent interacts with the repository through a set of structured tools rather than receiving pre-loaded context. Instead of being given relevant files directly, the agent must decide which repository information to access during the task.

To enable this interaction, an MCP server is designed to run as a TypeScript subprocess communicating over `STDIO`. The server is intended to expose a set of tools that allow the agent to explore and modify the repository. These tools include `list_files`, `read_file`, `search_in_files`, `run_tests`, `apply_patch`, and `git_diff`. Through these tools, the agent can inspect repository contents, execute tests, and apply code modifications during the repair process.

Each tool call is planned to be validated against a predefined schema before execution to ensure that requests follow the expected structure. This validation step is intended to prevent malformed or unsafe tool calls and to provide structured feedback to the agent when an operation cannot be completed.

D. Controlled Variables

Comparisons between strategies are only meaningful if everything else is kept fixed. Across all runs: we use the same LLM at the same temperature with the same sampling settings; the iteration limit and stopping conditions are identical (stop when all tests pass or the cap is reached); the execution environment is Python 3.12; and there is no external network access during runs. Each task-strategy combination is run multiple times to account for the inherent variability in model outputs.

VI. EVALUATION METRICS

We collect metrics in four areas. The first two are standard; the last two are less common but we think they matter.

A. Effectiveness

Did the agent fix the bug? We record the test pass rate after the agent’s final patch, a binary pass/fail outcome for each run, and pass@ k across multiple runs following the protocol from Chen et al. [1]. Pass@ k captures the probability that at least one of k attempts succeeds, which is meaningful when model variance is high.

B. Efficiency

A strategy that always fixes the bug but takes fifteen iterations and a million tokens per run is not practically useful. We record the number of repair iterations before success, wall-clock runtime per task, and total token usage broken down by input and output. These three together give a reasonable picture of operating cost.

C. Reliability

While success rate indicates whether a task was completed correctly, reliability metrics provide insight into the types of errors agents make during the repair process. To capture these behaviors, we track several failure signals that indicate unreliable or inconsistent agent actions.

Specifically, we monitor: (1) references to files that do not exist in the repository, (2) patches that cannot be applied successfully by the version control system, (3) edits made to modules that are unrelated to the failing test case, and (4) repeated actions that produce no change in the repository state.

These signals help identify situations where the agent is struggling to locate relevant context or generate valid modifications. Measuring such behaviors provides a more complete view of system performance beyond simple pass/fail task outcomes.

D. Strategy-Specific Signals

Some things only make sense to measure for one strategy. For RAG, we track retrieval coverage-specifically, what fraction of the retrieved chunks actually overlapped with the location of the injected fault. A low coverage score tells us the retrieval step was the bottleneck, not the generation step. For MCP, we track total tool call count per run and the rate of calls that fail schema validation or throw execution errors.

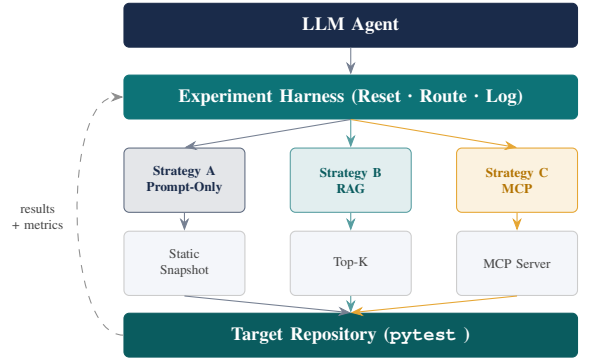


Fig. 1. System architecture of the LLM agent evaluation framework. All three strategies share the same harness, LLM, and target repository, isolating context-access mechanism as the sole experimental variable.

For prompt-only, we track how much of the available context window the static snapshot consumes, since saturation is the main failure mode for that strategy on larger repos.

VII. SYSTEM ARCHITECTURE

The evaluation framework consists of four primary layers: the LLM agent, the experiment harness, the three strategy implementations, and the target repository. Figure 1 illustrates the architecture and data flow.

A. LLM Agent

The agent sits at the top. It receives a prompt assembled by the harness and returns either a patch or, in the MCP case, a tool call request. The same underlying model configuration is used across all three strategies; the primary experimental difference lies in how repository context is surfaced to the model. This means any behavioral differences we observe come from the information structure of the prompt, not from the agent having different instructions.

B. Experiment Harness

The harness is the part we built specifically for this study. Before each run, it resets the repository via Git, applies the right fault patch, and then hands control to whichever strategy module is being evaluated. It enforces the iteration cap, catches the stopping condition when tests pass, and writes all collected metrics to a structured JSON log. The same harness code runs for all three strategies, so timing and logging are identical across conditions. This is the main thing that makes comparisons between strategies interpretable.

C. Strategy Implementations

Strategy A assembles a static file snapshot, injects it into the prompt, and does nothing else. Strategy B queries the index built at run initialization, takes the top- K results, and puts them in the prompt instead. Strategy C spawns the MCP server subprocess and connects the agent to it. The server enforces a read limit per file, rejects tool calls that fail schema validation, and logs every interaction. The agent drives the conversation from there like reading files, running tests, applying patches until it either passes or runs out of iterations.

D. Target Repository

All three strategies read from and write to the same repository directory. The harness’s Git reset at the start of each run guarantees that run $n + 1$ starts from exactly the same state as run n , regardless of what the previous agent did. Without that reset, accumulated state from a failed patch could make subsequent runs harder in ways that have nothing to do with strategy.

VIII. EXPECTED RESULTS AND DISCUSSION

Since the experiments have not yet been completed, this section outlines the expected outcomes and the reasoning behind them. These expectations are formulated based on insights from prior work.

H1 - Simple faults will not differentiate the strategies. For faults contained within a single function where the relevant file is obvious from the test name, all three strategies will probably reach the same answer in similar iteration counts. The prompt-only approach handles this case fine because there is not much context to navigate. We expect strategy differences to emerge on the two or three tasks where the fault requires reading across module boundaries [3].

H2 - MCP will produce fewer bad patches. Because every file access and patch application in Strategy C goes through a validated interface, the agent gets an explicit error response when it tries to do something structurally wrong. In Strategies A and B, there is no such feedback, the model either hallucinates a file path or generates an unparseable diff and finds out only when we run the tests. Based on Gorilla’s findings [7], we expect this to show up as a lower rate of failed patch applications in the MCP condition.

H3 - Prompt-only will cost the most tokens. Injecting a full file snapshot on every iteration is the least efficient possible use of the context window. RAG reduces this by only injecting retrieved chunks. MCP reduces this overhead because the agent reads only what it asks for. We expect token usage to follow the order: prompt-only > RAG > MCP, with the gap widening for tasks that require more back-and-forth.

H4 - RAG will be sensitive to retrieval quality. If the retrieval system does not bring the file that contains the bug into the prompt, the RAG agent will not have access to the necessary code to repair it. We plan to check this directly by measuring retrieval coverage on each task and correlating it with fix rate [5]. A low coverage score with a low fix rate would confirm that retrieval, not generation, was the bottleneck.

IX. CONCLUSION

The main question of this study is whether the way an LLM agent accesses repository context affects its ability to work with real software projects. Although this question may appear simple, there has been little controlled research directly comparing different context-access strategies. Most existing work on LLM-based code repair focuses on improving the

model itself or studies a single approach in detail without systematically comparing alternatives.

To address this gap, we developed an experimental framework that allows these strategies to be evaluated under controlled conditions. The framework includes a repository containing controlled repair tasks, failing-test correction scenarios, and feature-driven implementation tasks, along with an experiment harness that keeps all variables constant except the context-access strategy. In addition, we collect multiple evaluation metrics that measure not only whether tasks succeed, but also how efficiently and reliably the agent performs.

Once the experiments are completed, the results will help determine whether the choice of context-access strategy significantly affects repository-level LLM performance. If clear differences are observed, this would highlight the importance of designing effective context-access mechanisms. If the differences are small, the findings would still provide useful guidance for practitioners when deciding how much infrastructure is necessary for building LLM-based development tools.

REFERENCES

- [1] M. Chen, J. Tworek, H. Jun, Q. Yuan, H. P. de Oliveira Pinto, J. Kaplan, H. Edwards, Y. Burda, N. Joseph, G. Brockman, A. Ray, R. Puri, G. Krueger, M. Petrov, H. Khlaaf, G. Sastry, P. Mishkin, B. Chan, S. Gray, and W. Zaremba, “Evaluating Large Language Models Trained on Code,” 2021. Available: <https://arxiv.org/abs/2107.03374>
- [2] M. Tufano, C. Watson, G. Bavota, D. Poshyvanyk, M. Di Penta, and R. Oliveto, “An Empirical Study on Learning Bug-Fixing Patches in the Wild via Neural Machine Translation,” in *Proceedings of the 34th IEEE/ACM International Conference on Automated Software Engineering (ASE 2019)*, pp. 832–843, 2019. Available: <https://ieeexplore.ieee.org/document/9000077>
- [3] Z. Jiang *et al.*, “Impact of Code Language Models on Automated Program Repair,” 2023. Available: <https://ieeexplore.ieee.org/document/10172517>
- [4] Y. Gao, Y. Xiong, X. Gao, K. Jia, J. Pan, Y. Bi, Y. Dai, J. Sun, and H. Wang, “Retrieval-Augmented Generation for Large Language Models: A Survey,” 2023. DOI: <https://doi.org/10.48550/arXiv.2312.10997>
- [5] X. Zhang *et al.*, “RepoCoder: Repository-Level Code Completion Through Iterative Retrieval and Generation,” 2023. Available: <https://openreview.net/forum?id=oX14q5RRmF>
- [6] T. Schick, J. Dwivedi-Yu, R. Dessi, R. Raileanu, M. Lomeli, E. Hambro, L. Zettlemoyer, N. Cancedda, and T. Scialom, “Toolformer: Language Models Can Teach Themselves to Use Tools,” in *International Conference on Learning Representations (ICLR 2023)*, 2023. Available: <https://dl.acm.org/doi/10.5555/3666122.3669119>
- [7] S. G. Patil, T. Zhang, X. Wang, and J. E. Gonzalez, “Gorilla: Large Language Model Connected with Massive APIs,” 2023. DOI: <https://doi.org/10.48550/arXiv.2305.15334>
- [8] S. Yao, J. Zhao, D. Yu, N. Du, I. Shafraan, K. Narasimhan, and Y. Cao, “ReAct: Synergizing Reasoning and Acting in Language Models,” in *International Conference on Learning Representations (ICLR 2023)*, 2023. DOI: <https://doi.org/10.48550/arXiv.2210.03629>
- [9] R. Nakano, J. Hilton, S. Balaji, J. Wu, L. Ouyang, C. Kim, C. Hesse, A. Jain, V. Kosaraju, W. Saunders, X. Jiang, K. Cobbe, T. Eloundou, G. Krueger, K. Button, and J. Schulman, “WebGPT: Browser-Assisted Question-Answering with Human Feedback,” in *Advances in Neural Information Processing Systems (NeurIPS 2021)*, 2021. DOI: <https://doi.org/10.48550/arXiv.2112.09332>
- [10] C. E. Jimenez, J. Yang, A. Wettig, S. Yao, K. Pei, O. Press, and K. Narasimhan, “SWE-bench: Can Language Models Resolve Real-World GitHub Issues?” in *Advances in Neural Information Processing Systems (NeurIPS 2023)*, 2023. DOI: <https://doi.org/10.48550/arXiv.2310.06770>
- [11] Z. Liu *et al.*, “AgentBench: Evaluating LLMs as Agents,” 2024. DOI: <https://doi.org/10.48550/arXiv.2308.03688>